Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 5

на тему «Применение механизмов безопасности и обработки запросов в GraphQL API»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа № 5 посвящена аутентификации, сессиям и разграничению доступа в программном средстве анализа рентабельности продукции. Система объединяет производственный и финансовый учёт, рассчитывает себестоимость и прибыль, предоставляет GraphQL API и отчётность. Исправления бизнес-логики и инфраструктуры собраны в итоговой исполняемой версии lab7_profitability_system; прежние каталоги лабораторных сохраняют историю этапов разработки.

Сервисы владеют отдельными базами данных. Gateway проверяет пользователя и права, сервисный слой выполняет бизнес-операции, репозитории сохраняют данные только своего сервиса. В итоговой версии синхронный обмен выполняется по gRPC, а асинхронные реакции — через Kafka.

1 Auth Service и модель учётной записи

Auth Service использует identity.db с USER, ROLE и USER_ROLE. Пароли хранятся как PBKDF2-HMAC-SHA256 со случайной солью; открытый пароль в базу не записывается. Сервис предоставляет /auth/register, /auth/login, /auth/verify и /auth/logout. Gateway делегирует ему работу с учётными данными.

2 Регистрация без самостоятельного получения аналитических прав

Публичный register назначает только PENDING. Эта роль разрешает вход и получение сведений о своей учётной записи, но не чтение финансовой аналитики, калькуляций и отчётов, не запуск расчётов и импортов. Ни AuthService.register, ни базовый IdentityRepository.create_user не назначают ANALYST по умолчанию.

Бизнес-роль назначает оператор с доступом к доверенной среде приложения командой assign_role. Публичной Mutation самостоятельного назначения ролей нет. При следующей проверке Auth Service загружает актуальные роли из Identity, поэтому проверка доступа не опирается только на старые claims JWT.

```
python -m app.scripts.assign_role user@example.local ANALYST
# Команда выполняется оператором в доверенной среде приложения.
# Публичный register не принимает role и возвращает PENDING.
```

3 JWT и активная сессия

При login Auth Service проверяет пароль и статус ACTIVE, создаёт sid и Redis-сессию с TTL, затем выдаёт подписанный HS256 JWT. Проверяются подпись, issuer, type, срок действия и обязательные claims sub, sid, iat, exp. Наличие корректной подписи не заменяет активную сессию: отсутствие sid в Redis приводит к отказу. Logout удаляет сессию; последующее использование JWT отклоняется.

```
Client -> Authorization: Bearer <access JWT>
Gateway -> Auth Service /auth/verify
Auth Service -> signature, issuer, expiry, active session, current roles
GraphQL context -> Actor(user_id, email, roles, session_id)
ApplicationFacade -> authorization before service invocation
```

4 Матрица прав

Операция	ANALYST	PROD	FINANCE	MANAGER	ADMIN
products	Да	Да	Да	Да	Да
profitability / summary	Да	Нет	Нет	Да	Да
costBreakdown / allocations	Да	Нет	Нет	Да	Да
importRuns	Все	ERP/MES	FINANCE	Нет	Все
startProductionImport	Нет	Да	Нет	Нет	Да
startFinancialImport	Нет	Нет	Да	Нет	Да
retryImport	Нет	ERP/MES	FINANCE	Нет	Все
resolveExternalMapping	Нет	Да	Да	Нет	Да
calculateCost / profitability	Да	Нет	Нет	Нет	Да
exportReport	Свои	Нет	Нет	Свои	Свои

PENDING не имеет доступа к бизнес-операциям таблицы. ADMIN является суперролью для прикладных операций, однако экспорт сохранённого отчёта по умолчанию требует совпадения владельца; отдельное право чтения чужих сохранённых отчётов не вводится.

5 Разграничение журналов импорта

Query importRuns учитывает не только наличие одной из бухгалтерских ролей, но и направление загрузки. PRODUCTION_ACCOUNTANT без source получает только ERP/MES; FINANCE_ACCOUNTANT — только FINANCE. Запрос явно запрещённого source завершается AuthorizationError. Пользователь с обеими ролями видит оба направления. ANALYST и ADMIN могут просматривать оба контура.

Источник нормализуется и проверяется по допустимому набору. Для retryImport требуется бухгалтерская роль соответствующего найденного запуска. Запуск производственного и финансового импорта также проверяется отдельными ролями. Это устраняет чтение финансовых загрузок производственным бухгалтером и обратный доступ.

6 Владение SavedReport и ReportExport

exportReport не принимает requestedBy от клиента. Фасад передаёт user_id проверенного Actor. ReportingService.create_export сначала находит SavedReport, затем сравнивает SavedReport.user_id с requested_by. Только после успешной проверки создаёт задание и файл. По чужому reportId нельзя получить даже новую запись REPORT_EXPORT.

Query reportExport дополнительно проверяет requested_by перед возвратом статуса и file_ref. Идентификатор UUID сам по себе не является правом доступа. Несуществующий отчёт приводит к NotFoundError, чужой — к AuthorizationError, без побочного создания задания.

7 Защита внутреннего транспорта в итоговой версии

Порты gRPC 50051–50056 не публикуются на хост через Compose. Каждый RPC проходит InternalAuthorization: проверяются подписанный служебный JWT, issuer=profitability-internal, audience=internal-rpc, exp, sub и role=INTERNAL_SERVICE. Только доверенный gateway может инициировать изменяющие RPC; сервисы расчёта имеют доступ к межсервисному чтению. Пользовательские роли проверяются Gateway до выдачи служебного вызова.

Ключ INTERNAL_JWT_SECRET задаётся отдельно от пользовательского JWT_SECRET и обязателен при запуске самостоятельных gRPC-процессов. Эта граница защищает от прямого неаутентифицированного вызова внутренних Mutation. Внутренняя сеть и служебные ключи относятся к доверенной среде развёртывания.

8 Обработка ошибок и проверка

Проверка	Ожидаемый результат
Новая публичная регистрация	PENDING; запрос финансовой аналитики запрещён
Неверный пароль / истёкший JWT	AuthenticationError
JWT без активного sid	AuthenticationError
Logout и повтор запроса	Отказ после удаления сессии
PROD → source FINANCE	AuthorizationError
FINANCE без source	Только финансовые загрузки
Экспорт чужого SavedReport	AuthorizationError; число заданий не изменилось
RPC без служебного JWT	UNAUTHENTICATED
Изменяющий RPC от costing	PERMISSION_DENIED

Проверки выполняются в tests/test_auth.py, test_graphql_security.py и test_review_fixes.py итогового проекта. Тесты используют реальный gRPC server/client; для сессий допускается in-memory backend. Проверка состояния Redis и Kafka в Compose выполняется отдельно от этих тестов.

9 Запуск

```
# Скопировать .env.example в .env и задать случайный INTERNAL_JWT_SECRET.
docker compose build
docker compose run --rm auth python -m app.scripts.seed
docker compose up -d --wait
# GraphQL: http://127.0.0.1:8000/graphql
```

Заключение

Безопасность охватывает регистрацию, пользовательскую сессию, роль, направление импорта и принадлежность сохранённого отчёта. Пользователь не получает доступ к финансовым данным только за счёт регистрации или знания reportId. В итоговой распределённой версии внутренний gRPC также требует аутентификацию и проверяет роль служебного вызывающего компонента.